

Replicação AgentRx: Diagnóstico em Falhas de Sistemas Multiagentes utilizando Telemetria Estruturada

Pedro Henrique Silva Da Costa Gregório
pedro.gregorio@copin.ufcg.edu.br
Universidade Federal de Campina Grande (UFCG)
Campina Grande, PB, Brasil

Abstract

A depuração de Sistemas Multiagentes (MAS) baseados em Grandes Modelos de Linguagem (LLMs) é um problema relevante em *AgentOps*, pois esses sistemas operam de forma autônoma, não determinística e com múltiplos artefatos de execução ao longo da trajetória. O artigo AgentRx propõe um framework para diagnosticar falhas a partir dessas trajetórias, localizando a etapa crítica irrecuperável e classificando sua causa raiz a partir de restrições, evidências e julgamento via LLM. Entretanto, o estudo original depende de logs textuais heterogêneos, o que exige normalização complexa para uma representação intermediária. Neste trabalho, propomos uma replicação do AgentRx baseada em um MAS simulado e instrumentado com telemetria baseada no OpenTelemetry, comparando logs textuais simplificados e traces estruturados para avaliar a precisão do diagnóstico e da classificação de falhas.

Keywords

Sistemas Multiagentes, AgentOps, OpenTelemetry, Telemetria Estruturada, Injeção de Falhas, Diagnóstico de Falhas

ACM Reference Format:

Pedro Henrique Silva Da Costa Gregório. 2018. Replicação AgentRx: Diagnóstico em Falhas de Sistemas Multiagentes utilizando Telemetria Estruturada. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX)*. ACM, New York, NY, USA, 2 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 Introdução

Sistemas Multiagentes (MAS) baseados em Grandes Modelos de Linguagem (LLMs) vêm sendo empregados em tarefas cada vez mais complexas, combinando planejamento, uso de ferramentas e coordenação entre agentes. Essa autonomia, embora benéfica do ponto de vista funcional, amplia a dificuldade de observação e depuração, pois as execuções são probabilísticas, os fluxos são longos e os artefatos gerados ao longo da trajetória são variados [2]. Nesse contexto, AgentOps surge como um paradigma de observabilidade voltado a rastrear, correlacionar e analisar os artefatos produzidos por agentes ao longo de seu ciclo de vida, incluindo agente, plano, workflow, tarefa, ferramenta, avaliação e guardrails [2].

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference acronym 'XX, Woodstock, NY

© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/2018/06
<https://doi.org/XXXXXXX.XXXXXXX>

A literatura recente tem mostrado que a observabilidade de agentes não deve se limitar ao resultado final da tarefa, mas também ao processo de execução que leva a esse resultado. O estudo de Dong et al. [2] destaca que agentes de IA produzem artefatos complexos em diferentes camadas, e que esses artefatos devem ser rastreados por meio de traces, spans, eventos e métricas para permitir diagnóstico e responsabilização. Nessa direção, o OpenTelemetry fornece um vocabulário padronizado para representar execuções observáveis, incluindo atributos de spans, relações hierárquicas entre operações, eventos e métricas quantitativas associadas a custo, duração e uso de tokens [3].

Dentro desse cenário, o AgentRx propõe um framework de diagnóstico para localizar a primeira falha irrecuperável em trajetórias de agentes e atribuir sua causa raiz. Em sua formulação original, o método foi avaliado sobre 115 trajetórias falhas coletadas em três domínios reais e heterogêneos (*tau-bench*, *Flash* e *Magentic-One*), anotadas manualmente com a etapa crítica e a categoria da falha [1]. O fluxo do framework normaliza os rastros em uma Representação Intermediária (IR), sintetiza restrições globais e dinâmicas, produz um log de validação auditável e utiliza um juiz baseado em LLM para inferir a etapa crítica e a categoria da falha [1].

Partindo desse resultado, este estudo investiga se a mesma capacidade diagnóstica se mantém em um ambiente controlado e simulado de Sistemas Multiagentes, no qual as trajetórias são geradas experimentalmente e modeladas com telemetria estruturada inspirada em OpenTelemetry. Assim, a replicação compara dois cenários de representação das trajetórias: logs textuais simplificados e telemetria estruturada, buscando verificar tanto a replicabilidade do diagnóstico em um novo contexto quanto o efeito da estruturação dos registros sobre a qualidade da atribuição de falhas.

As questões de pesquisa que guiam este estudo são:

- **RQ1:** O AgentRx mantém sua capacidade de localizar a etapa crítica e categorizar falhas quando aplicado a um benchmark sintético gerado em um MAS simulado?
- **RQ2:** A telemetria estruturada melhora a identificação da etapa crítica da falha em comparação com logs textuais?

Neste contexto, este trabalho está organizado seguindo a estrutura: A Seção 2 apresenta o AgentRx, o modelo de telemetria proposto e a integração entre ambos. A Seção 3 descreve o desenho experimental, a instrumentação, as métricas e a análise estatística planejada.

2 Metodologia

2.1 AgentRx como baseline diagnóstico

O AgentRx recebe como entrada a trajetória falha, o conjunto de ferramentas e, quando aplicável, políticas de domínio. A partir desses

dados, o framework normaliza a execução em uma Representação Intermediária (IR), sintetiza restrições globais e dinâmicas e produz um log de validação com evidências passo a passo. Em seguida, um juiz baseado em LLM utiliza esse log, junto com um *checklist* taxonômico, para localizar a etapa crítica da falha e atribuir sua categoria [1]. Esse fluxo apresentado é a base metodológica reutilizada neste trabalho.

2.2 Ambiente experimental controlado

A replicação será conduzida em um MAS simulado, implementado com LangGraph, contendo quatro agentes: Coordenador, Pesquisador, Executor e Avaliador. As tarefas serão simples, porém com espaço para falhas de coordenação, falhas de ferramenta e falhas de validação. As ferramentas serão mockadas e localmente controladas, de forma a garantir reprodutibilidade e a permitir a injeção explícita de falhas.

2.3 Modelo de telemetria estruturada

O ambiente será instrumentado com um modelo de telemetria inspirado em OpenTelemetry, usando como entidades principais:

- **Resource:** execução completa do MAS;
- **Trace:** trajetória ponta a ponta da tarefa;
- **Span:** ação de um agente, chamada de ferramenta ou etapa de workflow;
- **Event:** ocorrência pontual, como delegação, retry ou timeout;
- **Metric:** tokens, latência, número de retries e custo estimado.

Cada execução armazenará os dados observáveis em formato estruturado, preservando atributos como agente, operação, status, erro, duração, tokens e relações hierárquicas entre spans. Essa telemetria servirá tanto como fonte da execução quanto como base para a geração de logs textuais simplificados, usados no cenário baseline.

2.4 Adaptação ao fluxo do AgentRx

Os traces gerados serão convertidos para o formato de IR esperado pelo AgentRx por meio de um adaptador. Essa camada transformará spans, eventos e atributos em uma trajetória compatível com o pipeline original, preservando o relacionamento entre passo, agente, operação e falha. O objetivo é manter o núcleo do diagnóstico do AgentRx, mudando apenas a forma de representação da execução.

3 Análise Experimental

3.1 Desenho experimental

O experimento seguirá um desenho fatorial completo 2×5 , considerando dois fatores: (i) tipo de representação da trajetória, com os níveis *logs textuais simplificados* e *telemetria estruturada*; e (ii) categoria da falha, com as classes *System Failure*, *Invalid Invocation*, *Instruction/Plan Adherence Failure*, *Tool Misuse* e *Validation Failure*. Cada combinação entre os níveis dos fatores será tratada como uma condição experimental independente.

Cada condição será executada cinco vezes, totalizando 50 trajetórias. A ordem das execuções será randomizada com *seed* fixa, e cada trajetória com falha será julgada três vezes pelo AgentRx para reduzir a variabilidade estocástica do LLM.

3.2 Variáveis e métricas

A variável independente é o tipo de representação da trajetória fornecida ao AgentRx. As variáveis dependentes são:

- (1) **Critical Step Accuracy:** Percentual de precisão na identificação do índice da etapa exata do erro [1].
- (2) **Failure Category Accuracy:** Precisão na classificação da categoria raiz [1].
- (3) **Average Step Distance:** Distância média, em passos, entre a predição do modelo e o erro real [1].

3.3 Análise estatística

A normalidade das métricas será verificada com o teste de Shapiro-Wilk. Caso a hipótese de normalidade não seja atendida, serão utilizados testes não paramétricos, em especial o teste de Wilcoxon pareado, com $\alpha = 0.05$. Além disso, serão reportados média, desvio padrão, coeficiente de variação e intervalos de confiança de 95% para as principais métricas.

3.4 Ground truth

O *ground truth* será definido automaticamente pelo operador de injeção de falha, que registrará a etapa em que ocorreu o primeiro desvio irreversível, a categoria da falha, o agente responsável e o instante da ocorrência. Essa estratégia elimina ambiguidade na anotação e permite comparar diretamente a saída do AgentRx com a falha efetivamente induzida.

References

- [1] Shraddha Barke, Arnav Goyal, Alind Khare, Avaljot Singh, Suman Nath, and Chetan Bansal. 2026. AgentRx: Diagnosing AI Agent Failures from Execution Trajectories. *arXiv preprint arXiv:2602.02475* (2026).
- [2] Liming Dong, Qinghua Lu, and Liming Zhu. 2024. Agentops: Enabling observability of llm agents. *arXiv preprint arXiv:2411.05285* (2024).
- [3] IBM. 2024. O que é OpenTelemetry? <https://www.ibm.com/br-pt/topics/opentelemetry>. Acessado em: 02 jun. 2026.